Index

S.No.	Title	Page No.
1	Introduction	2
2	Datasets Used	3-5
3	Literature Review	6-7
4	Project Structure	8-11
6	Literature Using Similar Architecture	12
5	Work Plan	13
6	Technologies Needed	13
7	Conclusion	14
8	References	15

Introduction

Modern AI assistants are usually powered by Large Language Models (LLMs). You can think of an LLM as a very well-read student who memorized a giant encyclopedia.

The catch is simple: the student's memory is frozen in time. If you ask about something new, private, or updated after training, the model does not truly "know" it. Sometimes it still answers confidently and can be wrong (hallucination).

Retrieval-Augmented Generation (RAG) fixes this by making the AI do an open-book test: it first searches an external knowledge base, then writes the answer using the retrieved evidence. The original RAG work describes this as mixing parametric memory (inside the model) with nonparametric memory (an external retrievable index).

Knowledge Base

A knowledge base in a Retrieval-Augmented Generation system is not just a collection of stored documents. It is a structured retrieval system that organizes information so the model can find the right evidence quickly, accurately, and with proper context. In modern AI systems, a knowledge base usually combines multiple retrieval styles such as lexical search, dense vector search, and sometimes graph-based relations, because each of them captures a different kind of relevance and helps reduce different failure cases.

In a generalized view, the knowledge base works through two connected lifecycles. The first is the offline lifecycle, where data is collected, cleaned, chunked, structured, represented, and indexed. The second is the online lifecycle, where a user query is understood, relevant candidates are retrieved, ranked, fused, and then used to generate a grounded answer. Around these two lifecycles, feedback, evaluation, governance, access control, and provenance ensure that the system stays reliable, updatable, and trustworthy.

Datasets Used

- BEIR
  Benchmarking-IR (BEIR) is a robust and heterogeneous evaluation benchmark for information retrieval. It has 18 publicly available datasets from a diverse range of text retrieval tasks and domains for this benchmark. On the BEIR benchmark, they evaluated 10 state-of-the-art retrieval systems, which encompassed lexical, sparse, dense, late-interaction, and re-ranking architectures.
  Paper - https://openreview.net/pdf?id=wCu6T5xFjeJ
  Data - https://github.com/beir-cellar/beir
  Documentation - https://www.tensorflow.org/datasets/catalog/beir

  Pre-processed Datasets it contains:
  1. Question Answering: MS MARCO, Natural Questions (NQ), HotpotQA, FiQA-2018
  2. Fact Checking: FEVER, Climate-FEVER, SciFact
  3. Bio-Medical IR: TREC-COVID, NFCorpus, BioASQ
  4. News & Tweet Retrieval: TREC-NEWS, Robust04, Signal-1M (RT)
  5. Duplicate Question Retrieval: CQADupstack, Quora
  6. Argument Retrieval: ArgAna, Touche-2020
  7. Entity Retrieval & Citation Prediction: DBPedia, SCIDOCS

beir/msmarco (default config)

- **Download size**: `1.01 GiB`
- **Dataset size**: `4.53 GiB`
- **Auto-cached** (documentation): No
- **Splits**:

Split	Examples
`'passage'`	8,841,823
`'query'`	509,962
`'test'`	9,139
`'train'`	516,472
`'validation'`	7,433

- **Examples** (tfds.as_dataframe):

Display examples...

	passage	passage_id	passage_metadata	query	query_id	query_metadata	score
0	The amethystine python is among the worldâ□□s biggest snakes, both in terms of length and weight. These giant snakes measure up to 8.5 meters in length and up to 90 kg in weight. Amethystine python is also known as scrub python. They found in tropical rainforests and grasslands of Australia and Papua Guinea.	1050419	{"title": ""}	most biggest snakes in the world	456505	{}	1.0
	1 Halibut is a very good source of magnesium, a natural calcium channel blocker, which lessens resistance and improves the flow of blood, oxygen and	1031106	{"title": ""}	is halibut healthy	411790	{}	1.0

- SituatedQA
  an open-retrieval Question Answering (QA) dataset where the system is required to provide the correct answer to a question, taking into account the relevant temporal or geographical context.This is a Context-Dependent Question Answering dataset, sourced from open-retrieval QA: Natural Questions, WebQuestions, TyDi-QA and MS-MARCO.

  All of these datasets are in English and contain questions that are answerable by Wikipedia documents.

  Question-context-answer triplets:
  Temp (train: 6009, dev: 3423, test: 2795)
  Geo (train: 3548, dev: 1398, test: 505).
  Paper - https://arxiv.org/pdf/2109.06157
  Data - https://github.com/mikejqzhang/SituatedQA
  Documentation - https://huggingface.co/datasets/siyue/SituatedQA

Use

```
from datasets import load_dataset
dataset = load_dataset("siyue/SituatedQA","temp","train")
```

Example:

```
{
  "question": "where will the next summer and winter olympics be held",
  "id": "2098168902147822379",
  "edited_question": "where will the next summer and winter olympics be held as of 2021",
  "date": "2021",
  "date_type": "sampled_year",
  "answer": ["Japan and China"],
  "any_answer": ["Brazil and S. Korea", "Japan and China"]
}
```

Literature Review

Dataset	Research Paper \ Open-Source	Structure Used
BEIR	An Analysis of Fusion Functions for Hybrid Retrieval(https://ar5iv.labs.arxiv.org/html/2210.11934)	Hybrid retrieval: fuses BM25 lexical scores (via PISA) with dense semantic scores from a dual-encoder (all-MiniLM-L6-v2) and studies convex-combination and reciprocal-rank fusion schemes
BEIR	A Dense Representation Framework for Lexical and Semantic Matching(https://ar5iv.labs.arxiv.org/html/2206.09912)	Introduces dense lexical representations (DLRs) by compressing high-dimensional lexical vectors through slicing and pooling; similarity is computed with a gated inner product and combined with dense semantic embeddings to form dense hybrid representations (DHRs)
SituatedQA	MRAG: A Modular Retrieval Framework for Time-Sensitive Question Answering(https://arxiv.org/pdf/2412.15540)	Training-free pipeline: an LLM splits each question into main content and temporal constraint; off-the-shelf dense retrievers (e.g., Contriever) fetch and summarize passages; sentences are re-ranked by multiplying a semantic similarity score and a symbolic temporal score
BEIR	SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval(https://arxiv.org/pdf/2109.10086)	Uses a BERT backbone with an MLM head to assign importance weights to vocabulary terms, producing sparse term-weight vectors via log-saturation; v2 improves pooling, offers a document-only expansion model and applies knowledge distillation
SituatedQA	Fusing Temporal Graphs into Transformers for Time-Sensitive Question Answering(https://arxiv.org/pdf/2310.19292)	Builds a directed temporal graph of events and times; fuses it into Transformer models either by tagging graph edges/nodes in the input (explicit edge representation) or using a relational graph convolution to update token embeddings

SituatedQA	MTGER: Multi-view Temporal Graph Enhanced Temporal Reasoning over Time-Involved Document(https://aclanthology.org/2023.findings-emnlp.1016.pdf)	Constructs a multi-view heterogeneous temporal graph with fact, time and global nodes; employs a heterogeneous graph attention network with adaptive fusion to reason over time-focused and fact-focused views, supplemented by a time-comparing objective and question-guided text-graph fusion

Project Structure

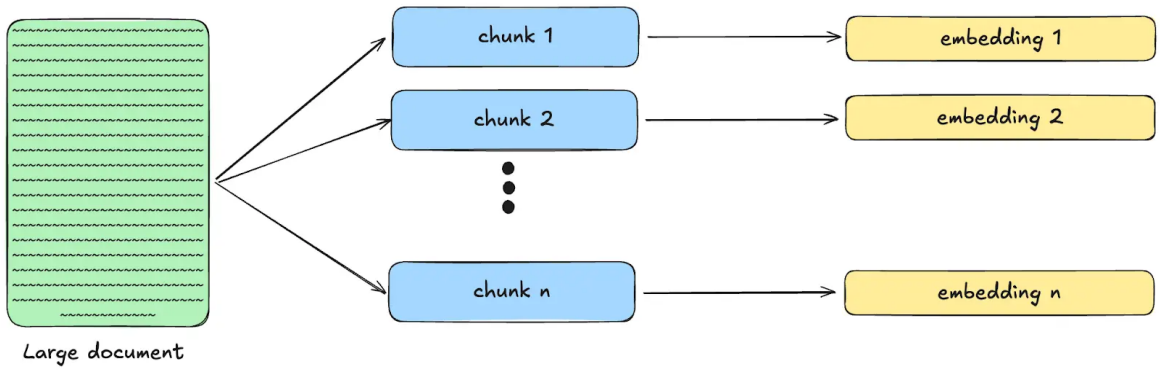
1. Query → Chunking → BM25 → Lexical Seeds → HNSW ANN → Dense Ranking

2. Query → Dense Encoder → Query Vector → HNSW ANN → Dense Ranking

3. BM25 Ranking and Dense Ranking → RRF Fusion → Final Results

Architectures:

1. Chunking
   Chunking is the process of breaking large documents into smaller pieces (typically) before creating embeddings and indexing them. Instead of having a 50-page text converted into a single embedding, chunking splits it into smaller logical units that can be searched over individually.

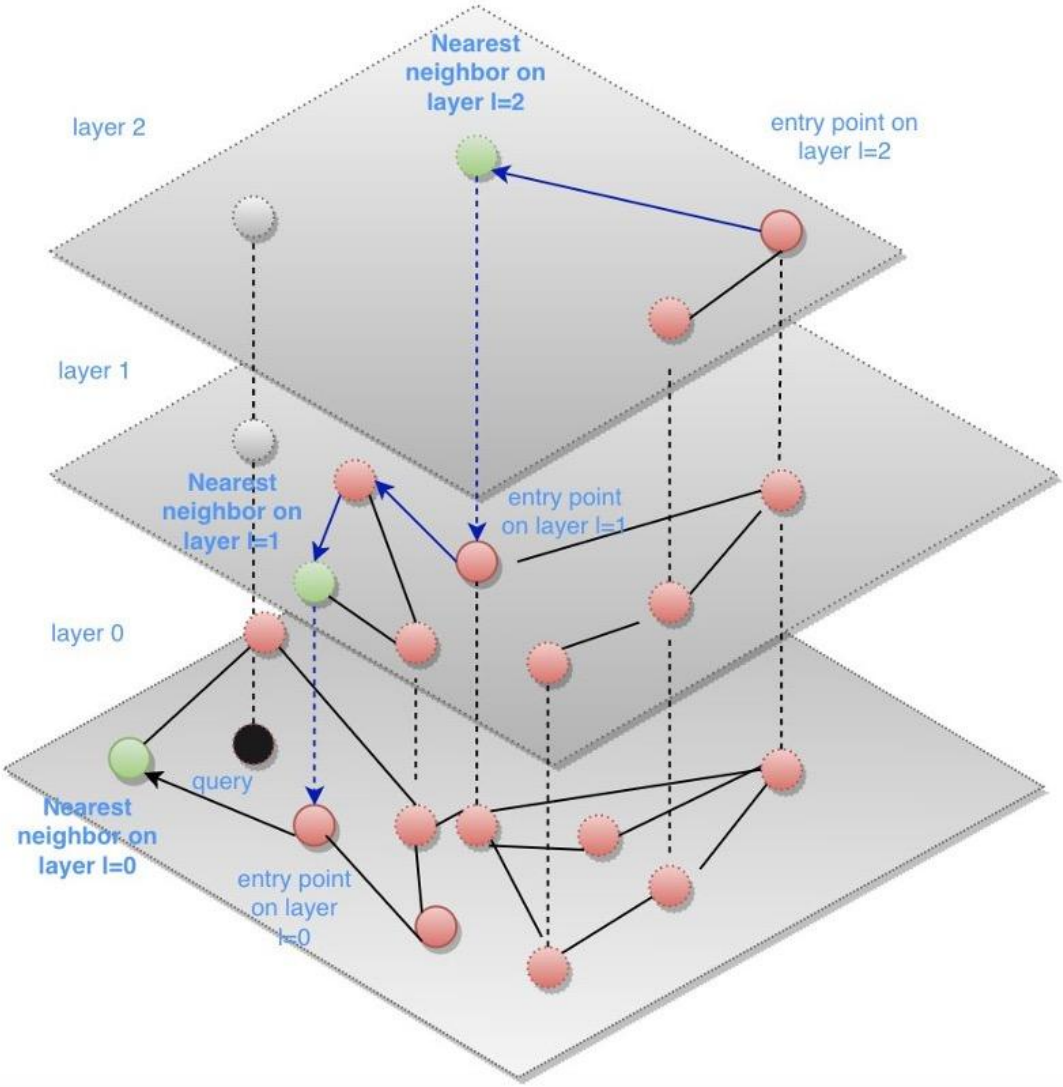


2. BM25
   BM25 (Best Matching 25) is a state-of-the-art ranking function used in information retrieval (e.g., Lucene, Elasticsearch, Solr) to rank matching documents by relevance to a query.

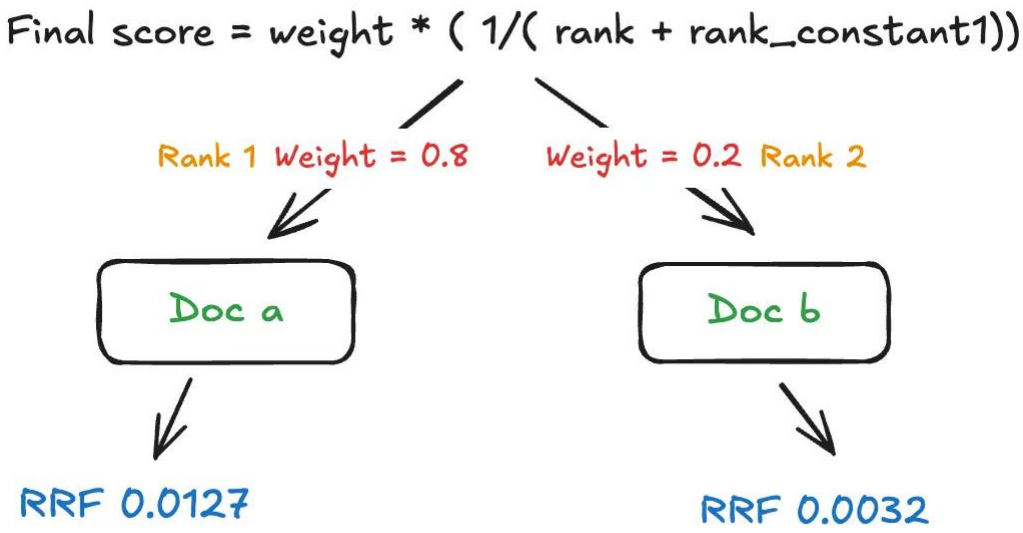
3. HNSW

   HNSW is based on similar principles as a skip list and the NSW algorithm. Its structure represents a multi-layered graph with fewer connections on the top layers and more dense regions on the bottom layers.

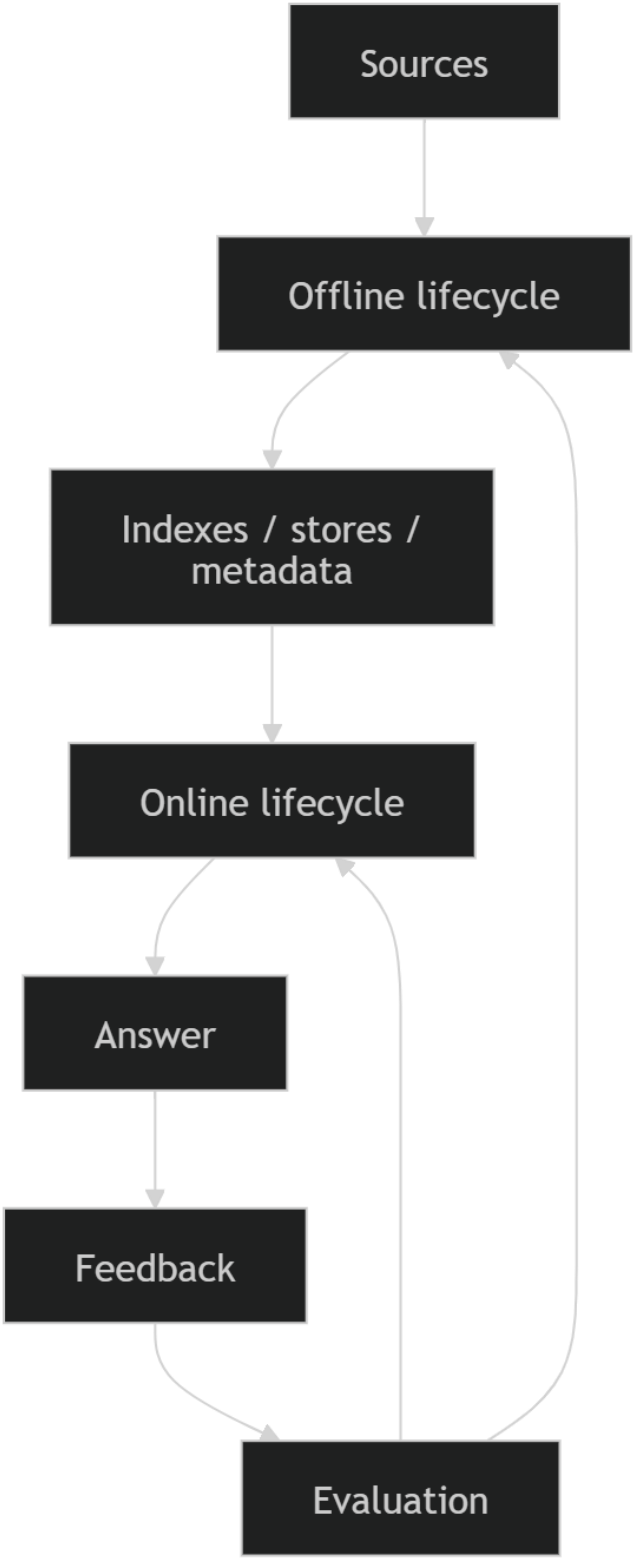
   The following figure shows how the search starts from the highest layer and proceeds to one level below every time the local nearest neighbor is greedily found among the layer nodes. Ultimately, the found nearest neighbor on the lowest layer is the answer to the query.



4. RRF Fusion


Final score = weight * (1/(rank + rank_constant1))
Rank 1 Weight = 0.8
Weight = 0.2 Rank 2
Doc a
Doc b
RRF 0.0127
RRF 0.0032

General Pipeline:



Literature using similar Architecture

1. Lexically-Accelerated Dense Retrieval
   https://arxiv.org/pdf/2307.16779
   LADR seeds an ANN graph with BM25 results and explores neighbors proactively or adaptively. It shows large recall improvements at low latency: e.g. at ~8ms per query, LADR "consistently achieves precision and recall on par with exhaustive search"
2. LexBoost
   https://arxiv.org/html/2409.05882v1
   LexBoost precomputes a dense neighbor graph offline and boosts BM25 scores with neighbors' BM25 scores. Unlike LADR, LexBoost performs only lexical scoring online. It reports significant improvements over BM25 on TREC DL2019 by leveraging neighbors found by a dense model.

Work Plan

1. Build BM25 index and test lexical baseline
2. Generate dense embeddings and build HNSW
3. Implement parallel hybrid retrieval with RRF
4. Develop lexically seeded HNSW and tune ANN parameters
5. Add metadata filtering
6. Run experiments and collect metrics
7. Perform ablation studies
8. Analyze results and prepare final report/demo

Technologies Needed

- Python 3.10+
- NumPy 2.4.2
- PyTorch 2.10
- Apache Lucene 10.3.1
- Sqlite3
- Beir benchmarks

Conclusion

Hybrid knowledge bases represent a necessary evolution in modern retrieval systems because no single representation of information is sufficient for all query types and reasoning needs. By combining lexical, semantic, relational, and metadata-aware retrieval mechanisms, a hybrid knowledge base provides a more robust, context-sensitive, and scalable foundation for information access. As AI systems increasingly depend on external knowledge that must remain current, verifiable, and domain-specific, the role of such knowledge bases will become more central. In the coming years, hybrid knowledge bases are expected to serve as a foundational layer for intelligent applications that require reliable retrieval, adaptable memory, and structured interaction with continuously growing information.

References

1. https://aws.amazon.com/blogs/database/optimize-generative-ai-applications-with-pgvector-indexing-a-deep-dive-into-ivfflat-and-hnsw-techniques/
2. https://www.elastic.co/search-labs/blog/chunking-strategies-elasticsearch
3. https://www.elastic.co/search-labs/blog/weighted-reciprocal-rank-fusion-rrf
4. https://github.com/beir-cellar/beir?tab=readme-ov-file
5. https://www.tensorflow.org/datasets/catalog/beir
6. https://huggingface.co/datasets/siyue/SituatedQA
7. https://learn.microsoft.com/en-us/azure/search/hybrid-search-ranking